Turing Machines

There was a time, not too long ago, when people had not yet precisely defined what computation was. Alan Turing took on this task. Here is what he came up with.

CONTENTS

The Backstory • The Basic Idea • Thirteen Examples • Formal Definition • More Examples • Variations on the Turing Machine • An Online Simulator • The Universal Machine • Busy Beavers • Summary

The Backstory

History helps a lot! Why do we have Turing Machines? Why do we care? Why are we better knowing about Turing Machines than not knowing them?

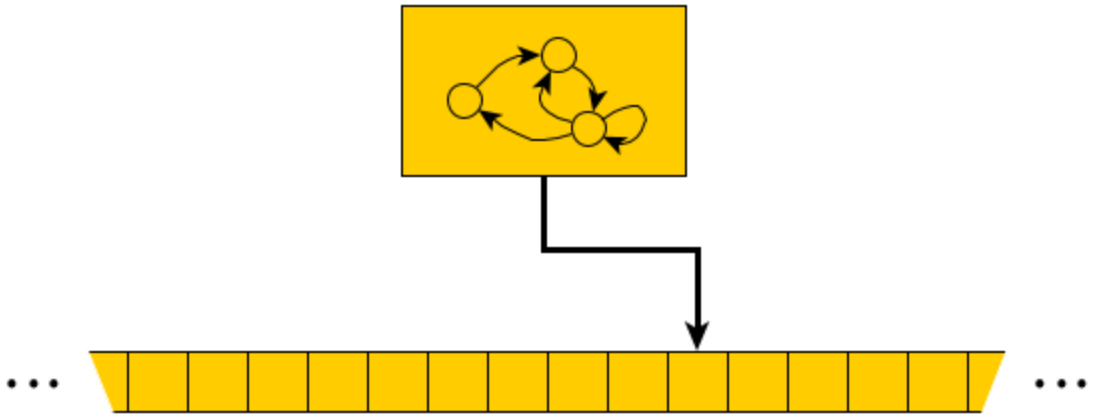
Hear the answers in class. We'll go through 19th century advances in logic, the formalists versus the intuitionists, Hilbert's Challenges, the need to formalize computation, Gödel's and Church's approaches, and how Turing's view of computation was the most mechanistic (as opposed to logical or mathematical) and influenced by human computers of the day, and how he (Turing) translated all these ideas into simple mathematical constructs that in some real sense....nailed it!

The Basic Idea

Turing looked at computers of his day and asked "What are these people doing?" He thought about it and realized these folks would write something down, think for a bit, write something else or erase something, think some more, write or erase, and so on. They would move from mental state to mental state as they worked, deciding what to do next based on what mental state they were in and what was currently written.

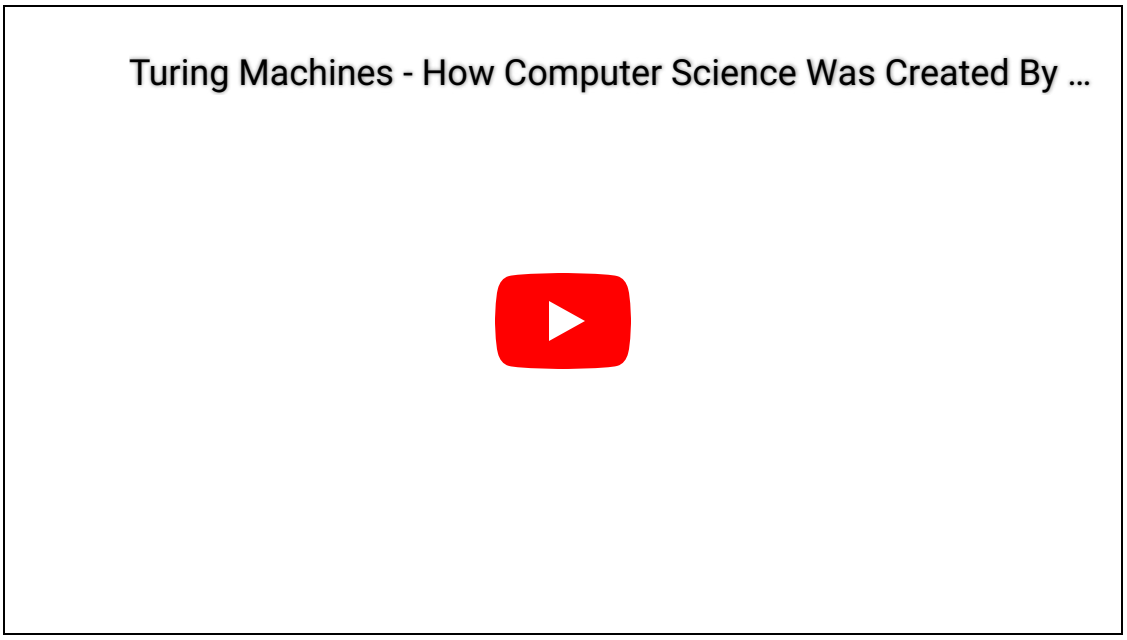
He realized he could capture this process formally and keep the model very simple: instead of a two-dimensional page, a single one-dimensional tape would do, and instead of randomly jumping around, it would be okay to move one cell at a time. Today we picture the machines like this:





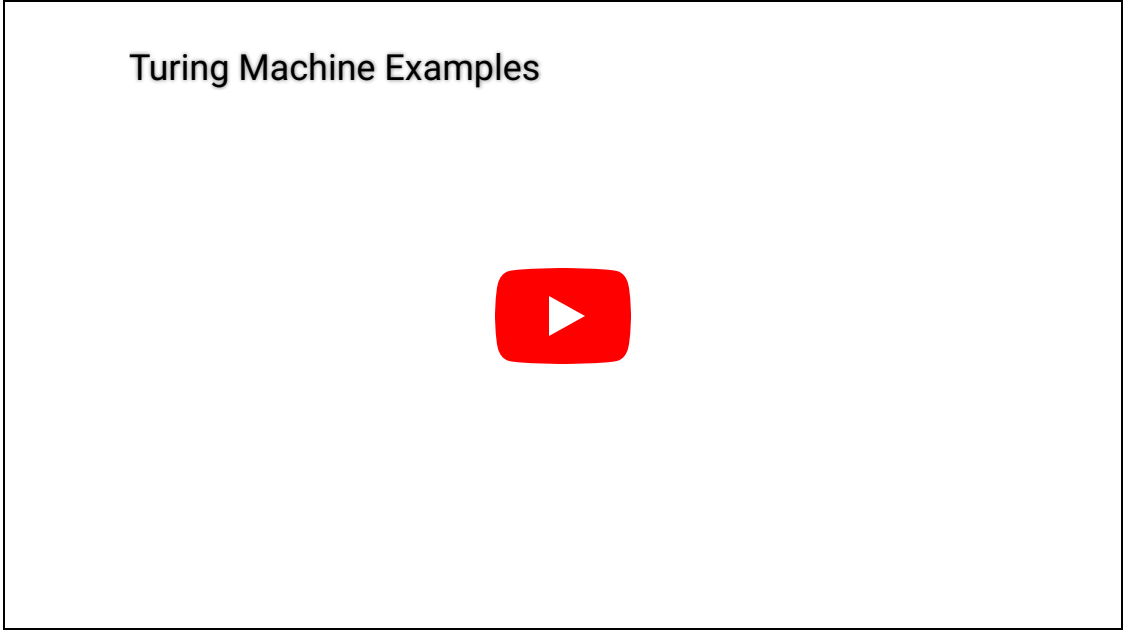
SIngle infinite tape for input and output

Jade's video, again:



Thirteen Examples

Let's see some examples before we get formal.



Formal Definition

Y'all set for a formal definition? Here is the definition for a transducing TM:

> A **Turing Machine** is a tuple $(Q, \Sigma, \Gamma, b, R, S)$ where:
>
> - Q is a finite set of states
> - Σ is the alphabet of symbols that comprise our inputs and outputs
> - Γ is the symbols allowed to appear on the tape
> - $b \in \Gamma - \Sigma$ is the blank symbol
> - $R \subseteq (Q \times \Gamma) \times (\Gamma \times \{\mathsf{L}, \mathsf{R}\} \times Q)$ says what can be done in a given state looking at a given symbol
> - $S \in Q$ is the distinguished start state
>
> A **configuration** Ψ is a tuple $(u, q, v) \in \Gamma^* \times Q \times \Gamma^*$ describing a machine with uv on the tape preceded by, and followed by, an infinite number of blanks, with the machine looking at the first symbol of v.
>
> The **transition relation** $\Longrightarrow$ on configurations relates configurations with the configurations they give rise to after one **step** of execution. Formally (note $u, v \in \Gamma^*$ and $\sigma, \sigma' \in \Gamma$):
>
> $$\begin{aligned} \Longrightarrow_R \quad &= \quad \{((u, q, \sigma v), (u\sigma', q', v)) \mid (q, \sigma)\, R\, (\sigma', \mathsf{R}, q')\} \\ &\cup \quad \{((u\sigma, q, v), (u, q', \sigma' v)) \mid (q, \sigma)\, R\, (\sigma', \mathsf{L}, q')\} \end{aligned}$$

> The machine $(Q, \Sigma, \Gamma, b, R, S)$ **computes** w' from w if $w, w' \in \Sigma^*$ and
>
> $$\exists\, u\, q\, \sigma\, v.\ (\varepsilon, S, w) \Longrightarrow_R^* (u, q, \sigma v) \wedge w' = \mathsf{trim}(u\sigma v) \wedge \nexists \Psi.\ (q, \sigma)\, R\, \Psi$$

What about the recognizer flavor? It's just this:

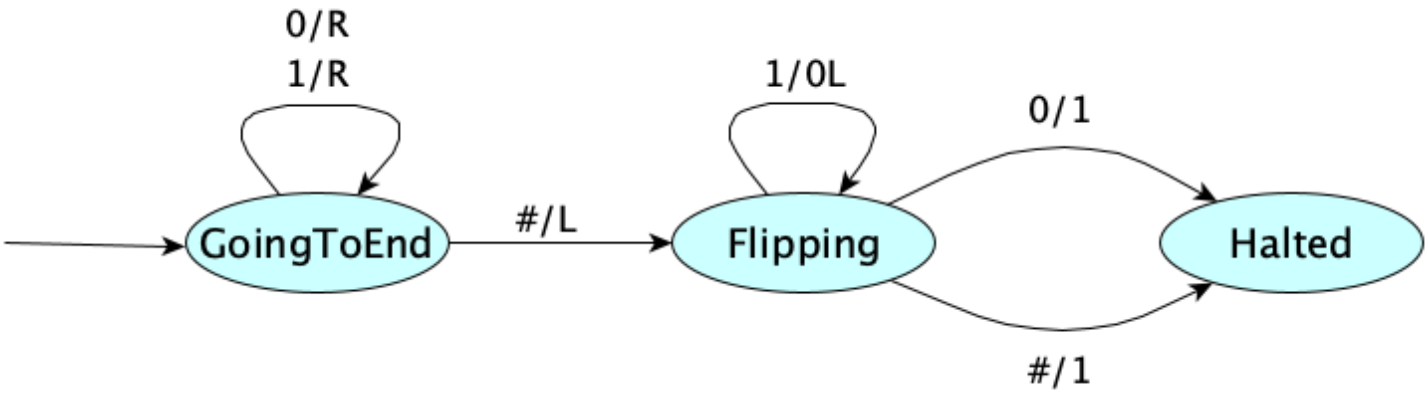
> A **Recognizing Turing Machine** is a tuple $(Q, \Sigma, \Gamma, b, R, S, A)$ where:
>
> - $(Q, \Sigma, \Gamma, b, R, S)$ is as defined above
> - $A \subseteq Q$ is a distinguished subset of **accept states**
> - Configurations and the transition relation are as above
>
> The machine $(Q, \Sigma, \Gamma, b, R, S, A)$ **accepts** the string w if $w \in \Sigma^*$ and
>
> $$\exists u\, q\, \sigma\, v.\ (\varepsilon, S, w) \Longrightarrow_R^* (u, q, \sigma v) \wedge q \in A \wedge \nexists \Psi.\ (q, \sigma)\, R\, \Psi$$
>
> The set of all strings accepted by machine M is said to be $L(M)$, the language **recognized** by M.

***Exercise**: Study the definitions above until you understand everything about them and can regenerate them without looking. Don't just memorize. Understand.*

> ***There is more than one kind of Turing Machine***
>
> *The definition above is the one I like best.*
>
> *In this definition, the transition relation was allowed to be any relation whatsoever. In fact, it is* **non-deterministic***: at each step, the machine may have multiple choices regarding what to next! And the machine runs until it hits a configuration with nowhere to go.*
>
> *We will see variations later, including* **deterministic** *models which require the transition relation to be a function, and a special* HALT *state, or* HALT *action, to be introduced.*

Case Study

Let's use the formal definition to describe a Turing Machine that increments a binary number. Here's the machine as a picture:



Let's identify the parts. Our machine is $(Q, \Sigma, \Gamma, \#, \mathsf{R}, \mathsf{G})$ where:

> $Q = \{\mathsf{G}, \mathsf{F}, \mathsf{H}\}$
> $\Sigma = \{0, 1\}$
> $\Gamma = \{0, 1, \#\}$
> $R = \{$
> $((\mathsf{G}, 0), (0, \mathsf{R}, \mathsf{G})),$
> $((\mathsf{G}, 1), (1, \mathsf{R}, \mathsf{G})),$
> $((\mathsf{G}, \#), (\#, \mathsf{L}, \mathsf{F})),$
> $((\mathsf{F}, 1), (0, \mathsf{L}, \mathsf{F})),$
> $((\mathsf{F}, 0), (1, \mathsf{L}, \mathsf{H})),$
> $((\mathsf{F}, \#), (1, \mathsf{L}, \mathsf{H})),$
> $\}$

We can condense this a bit, using only the rule set, making sure the start symbol appears first (spaces added for clarity):

```
G00RG G11RG G##LF F01LH F#1LH
```

You can come up with a canonical encoding, where states are 0, 1, 2, ...; Σ is $\{0, 1\}$; the symbols in $\Gamma - \Sigma$ are 2, 3, 4, ... with the blank being 2; Left is 0 and Right is 1:

```
00010 01110 02201 10102 12102
```

More Examples

If the 13 examples in the video were not enough, we have more.

Binary Decrement

TODO

Reverse a String

TODO

Binary Addition

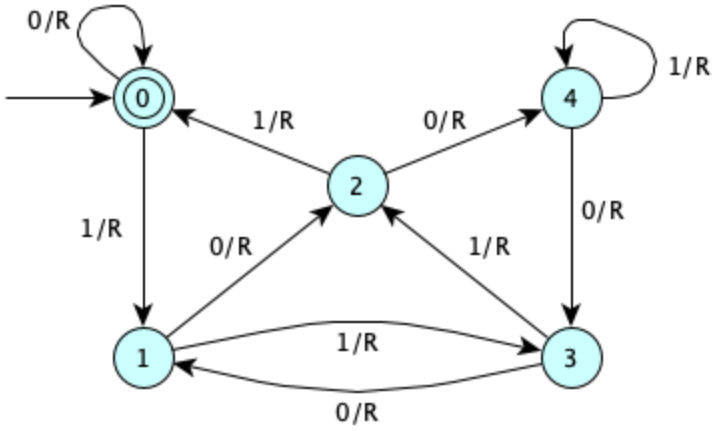
TODO

Parity

TODO

Is Divisible by 5?

Take the empty string to be 0. Read the binary string from left to right. Whenever you see a zero, your current number goes from n to $2n$. Whenever your see a 1, your number increases to $2n + 1$. Keep track of the current number modulo 5. You accept if you finish with a number whose value mod 5 is 0.



Even More Examples

Want even more examples? Check out this article with six interesting examples, including:

- Binary addition
- Converting binary to unary
- Converting unary to binary
- Doubling the length of a sequence
- The 4-state busy beaver
- Palindrome detection

Variations on the Turing Machine

Let's mix things up a bit.

Determinism

We've defined our machines with a transition relation that could have multiple outgoing transitions from a state for the same configuration. But what if we required the machine to have, for every state, one and only one transition? That is, instead of:

$$R \subseteq (Q \times \Gamma) \times (\Gamma \times \{L, R\} \times Q)$$

we required R to be a function? Doing so would require us to expand the codomain of the function so that the machine could stop. We could add a **halting action**:

$$R : (Q \times \Gamma) \to (\Gamma \times \{L, R\} \times Q) \cup \{\text{HALT}\}$$

This says: if the machine is in some state $\sigma \in Q$, and currently looking at a symbol $\sigma \in \Gamma$, then either:

1. Replace the symbol with a new symbol σ', move one cell, and enter state $q' \in Q$, OR
2. Halt

A slightly different definition of a deterministic Turing Machine treats "halting" as a kind of state, a **halting pseudo-state**:

$$R : (Q \times \Gamma) \to (\Gamma \times \{L, R\} \times (Q \cup \{\text{HALT}))\}$$

This says: if the machine is in some state $\sigma \in Q$, and currently looking at a symbol $\sigma \in \Gamma$, then:

1. Replace the symbol with a new symbol σ', move one cell, THEN
2. Either enter state $q' \in Q$ or Halt

> ***Exercise**: Study and appreciate the subtle differences between these definitions. Do you feel your mathematical maturity rising?*

CLASSWORK

How many n-state deterministic TMs are there?

Recall from your discrete math class that the number of functions over finite domain D and finite range R is $|R|^{|D|}$ — you did pay attention in that class, right? Haha, it's okay, I had forgotten that too.

We will show that, for the halting-pseudo-state version, the domain has size $2n$ and the range has size $4(n+1)$.

▶ Hint

Later we will see why this is interesting.

The definition is a little clunkier, but it's kind of nice. Interestingly, this simplification does not change the computing power of the model.

Stationary "Moves"

Instead of having to go left or right at each step, you can also provide to option to stay put. No change to computing power.

$$R \subseteq (Q \times \Gamma) \times (\Gamma \times \{\mathsf{L}, \mathsf{R}, \mathsf{X}\} \times Q)$$

Here X means stay on the same cell.

ε-Moves

In the base model, the machine looks at the current symbol and decides what to do next. We can let it just decide to jump to another state without examining the current symbol. Again, no change in computing power.

$$R \subseteq (Q \times (\Gamma \cup \{\varepsilon\})) \times (\Gamma \times \{\mathsf{L}, \mathsf{R}\} \times Q)$$

Multidimensional Tapes

How about, instead of a tape that is infinite only in the left and right directions, we make a two dimensional tape that can go up and down also? Or a 3-D tape? Or make a machine whose tape is any finite number of dimensions? All good. These extensions do not increase the computing power of the model.

One-way Infinite Tape

What if the tape had a left-end, and was infinite only to the right? A little messier to program, perhaps, but no change in computing power.

Multiple Tapes

More than one tape? More convenient. But you guessed it, no change in computing power.

Stack and Queue Tapes

If we restricted our tape memory to act like a queue, we have a Queue Automaton and no change in computing power. But if our tape was stack-like, we've reduced our power: we can no longer compute certain functions (recognize certain languages). But if we had **two stacks** we are back to full power.

Read-Only and Write-Only and Append-Only Tapes

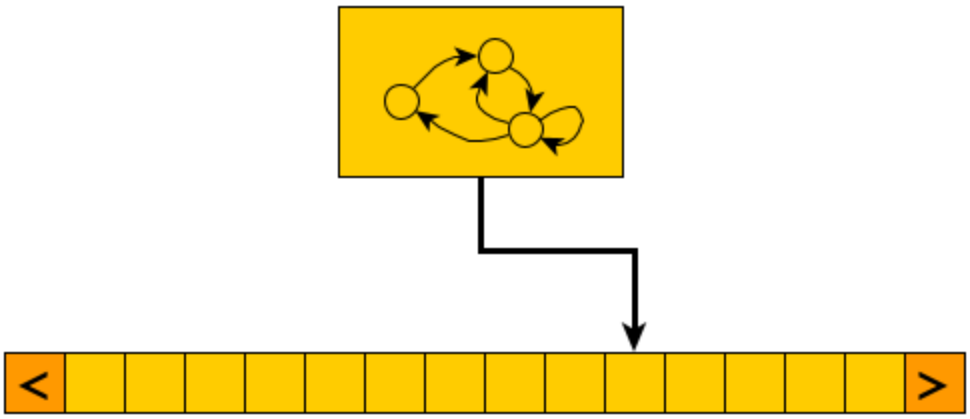
Sometimes we might want to have a bunch of tapes and use certain tapes only in certain ways. Maybe we can only read or only write or only append. Hmmm, tapes are starting to look flexible. We can imagine a tape that is:

- Unbounded on both ends, bounded on one end and unbounded on the other, bounded on both ends
- 1-dimensional, 2-dimensional, ... n-dimensional
- Read-only, Write-only, Append-only

- Stack-like, Queue-like, Move one cell at a time, Take big jumps
- One way (say right-moves only or left moves only) or multi-way

Linear Bounded Automata (LBAs)

What if we bounded the tape on both ends? There's this thing called a **Linear Bounded Automaton** which is a TM with a bounded tape, just big enough to hold the input and two additional special tape symbols, the left-end-marker and the right-end-marker. These symbols can not be overwritten, and you cannot move beyond them. They are super convenient because how else would keep from falling off the ends?



Fixed-size tape bounded to input string plus two cells for endmarkers

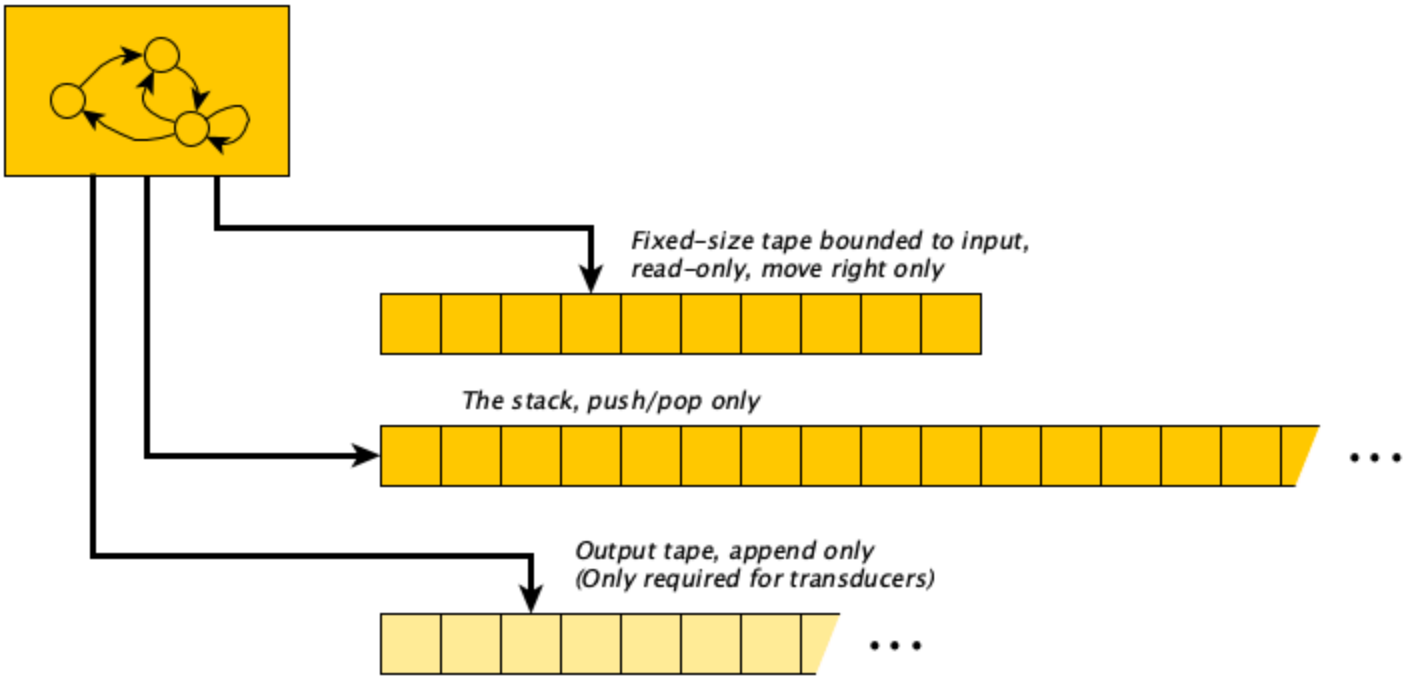
Any change in computing power? It turns out yes. The computing power is reduced.

> ***Exercise**: (Research) Find a language recognizable by a Turing Machine but not by an LBA.*

Pushdown Automata (PDAs)

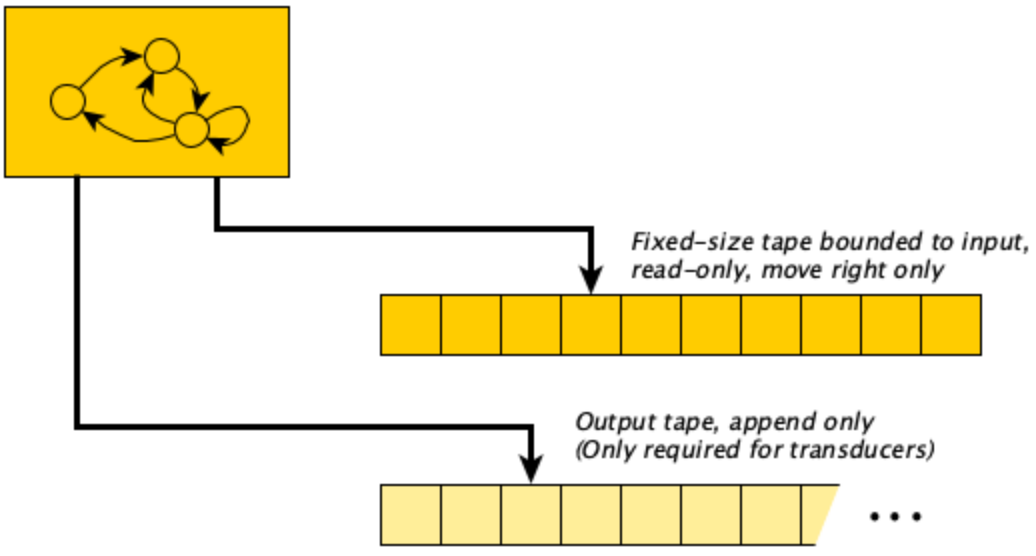
It turns out that the class of context-free languages can be decided exactly by a machine with two tapes: (1) a bounded, read-only, right-move only table holding exactly the input, and (2) a stack-like, one-way infinite tape for whatever you need. This kind of machine is called a push-down automaton (PDA). It is useful because stacks are super efficient, $\Theta(1)$ for pushing and popping. The PDA runs until there is no transition to make, or until the machine exhausts its input.

If you want a PDA transducer, add an output tape.



Finite Automata (FAs)

What if we had no memory at all? We'd have something called a **Finite Automaton**.



A lot of computations fall into this category.

An Online Simulator

When writing a Turing machine to carry out a computation or decide a question, don't just guess! Use a real online simulator to check your work!

> ***Exercise**: Write your own simulator.*

The Universal Machine

The most brilliant idea ever.

OMG.

Check this out. Remember that Turing Machine we saw above to increment a binary number? Remember how it could be encoded as a string:

```
0001001110022011010212102
```

Every Turing Machine can be represented as a string. Turing Machines take strings as input. Wouldn't it be cool if we could make a single TM that could read the description of a machine M, along with some other input w, and simulate M running on w?

Turing showed this was possible.

A machine that that do this is called a **Universal Turing Machine**.

The existence of a UTM shows that general purpose computers are possible. Indeed Turing's work predated real general purpose computers.

To see how Turing found the Universal Machine, start at Wikipedia.

Busy Beavers

An **n-state busy beaver** is a deterministic n-state, halting, Turing Machine with $\Sigma = \{1\}$ and $\Gamma = \{b, 1\}$ that writes the largest number of 1s on an initially blank tape over the set of all such n-state, halting Turing Machines, using the *second* definition of deterministic Turing Machine above (the kind that is allowed to write a symbol before halting).

Why on Earth would be care about such things? Let's find out. First, let's see what we are up against. Previously we saw that the number of deterministic n-state Turing Machines is $(4(n+1))^{2n}$. Hey that's a lot of Turing Machines! For fun, here are the first few values:

n	$(4(n+1))^{2n}$
0	1
1	64
2	20736
3	16777216
4	25600000000
5	63403380965376
6	232218265089212416
7	1180591620717411303424
8	7958661109946400884391936
9	68719476736000000000000000000

For a given n, some of these machines will halt, and some will not. Now we can go through all the halting ones (haha) and count the number of 1s they print out. The maximum number we find is $\Sigma(n)$. Let's see what these values are:

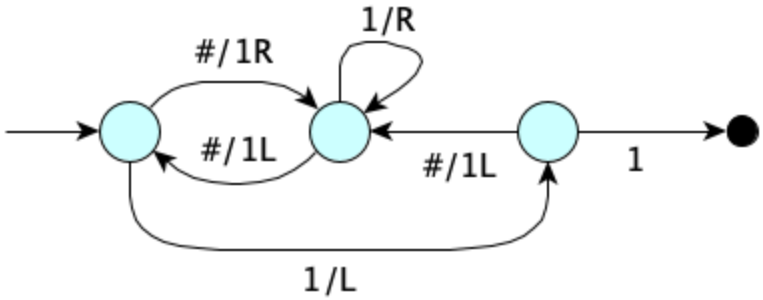
n	$\Sigma(n)$
0	0
1	1
2	4
3	6
4	13
5	at least 4098
6	greater than 3.515×10^{18267}
7	greater than $10^{10^{10^{10^{18705353}}}}$

Woah.

It seems amazing that we *know* what $\Sigma(4)$ even is! Knowing this means that for each of the 25.6 BILLION 4-state, 2-symbol Turing Machines, someone figured out whether it halts or not, and if it halts, what it outputs. Finding $\Sigma(5)$ will be harder: there are over 63.4 TRILLION machines to investigate.

That's a lot of machines to go through. And it's not easy to figure out whether a long-running machine will eventually halt or not. Such a machine might take years or centuries to finish, so we can't just run it and see. We have to painstakingly analyze each machine, and we have to sometimes be really clever. And if we know a long running machine will halt, we need a clever way to figure out how many 1s it will output, because this number might be too big to actually store. Can you see now why people that try to find these values are called **Busy Beaver Hunters**?

Here is a 3-state Busy Beaver you can try out:



You can read more about Busy Beavers at OIS and at Wikipedia.

> ***More Busy Beaver Fun To Follow***
>
> *When we study computability theory, we'll see that Σ is not a computable function. Although it has a well-defined finite description, no computing device can compute it. In fact, it grows faster than any computable function.*
>
> *Imagine that.*

Impatient? Here is a very nice short paper with formal proofs that Σ, and its friend S (function for the maximum number of *steps* a 2-symbol, n-state DTM takes), are uncomputable.

Here's more on Busy Beavers:

The Boundary of Computation



Summary

We've covered:

- ✅ Historical motivation
- ✅ How Turing viewed computation
- ✅ Lots of Examples
- ✅ Formal Definition
- ✅ Variations
- ✅ Universality
- ✅ Busy Beavers

Register Machines

Tape-based automata, like the Turing Machine, are not the only formal models of computation studied in computer science.

CONTENTS

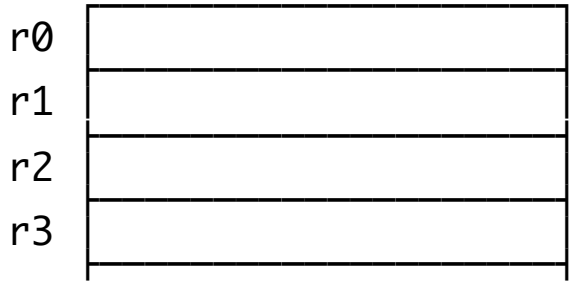
Why Registers? • The Basics • Counter Machines • RAMs • RASPs • Summary

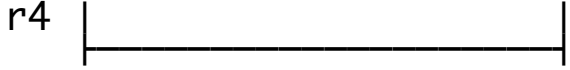
Why Registers?

Tape-based automata are quite minimalistic, and great for making a theory from just about the most primitive concepts. Registers feel a little higher-level, and also, let's face it, real machines have registers.

The Basics

In automata theory, A **register machine** has a number of registers $r_0, r_1, r_2, \ldots$, each holding a single (unbounded) natural number. The machine's control is a program, defined as a list of (labeled) instructions. The instructions are normally executed in sequence, but some instructions can cause a "jump" to an arbitrary instruction. The instruction set varies from model to model (of which there exist an infinite number of possibilities).





Don't miss the Wikipedia article on register machines.

The simplest kind of register machines are those whose instruction set only allows registers to be incremented or decremented only. These are called **counter machines**.

Counter Machines

Here's a sample instruction set (I just made it up) for a counter machine:

Various Counter Machine Instructions	
CLEAR r_i	$r_i := 0$
INC r_i	$r_i := [r_i] + 1$
DEC r_i	$r_i := [r_i] - 1$
COPY r_1, r_j	$r_j := [r_i]$
JZ r_i, L	If $[r_i] = 0$ then jump to L
JE r_i, r_j, L	If $[r_i] = [r_j]$ then jump to L
JZDEC r_i, L	If $[r_i] = 0$ then jump to L else $r_i := [r_i] - 1$
JUMP L	Jump to instruction at address L

This instruction set is pretty simple; it assumes the input is the contents of the registers before execution, and the output is the contents of r_0. The program is a list of instructions and the program ends when you have no more instructions.

For convenience, we can also imagine a machine that has separate streams for input and output, operated on by READ and WRITE instructions, respectively. We might even throw in a convenient instruction to halt the program:

More Instructions	
READ r_i	Consume next value from input stream and store in r_i
WRITE r_i	Append contents of r_i to output stream
HALT	Stop executing

Because counter machine programs are primitive, like Turing Machines, programs are pretty long. Here's a program that reads to integers, adds them, and writes their sum.

```
0 |  READ r0           // x
1 |  READ r1           // y
2 |  JZDEC r1, 5       // if y is 0, we're done
3 |  INC r0            // x++
4 |  JUMP 2            // loop back
5 |  WRITE r0          // there's the output
```

If you have the right set of instructions, you can show a register machine to be equivalent in computing power to a Turing machine.

Surprisingly, there exists a register machine that is equivalent in computing power to a Turing Machine that has only two instructions:

Simplest Counter Machine	
INC r_i	Increment register r_i
JZDEC r_i, L	If register r_i holds 0, jump to instruction at address L, otherwise decrement register r

> ***O RLY?***
>
> *Remember the reads and writes aren't actually needed. You can still compute by having a convention such that the input of the program starts in register 0 and the output is read from register 1 whenever there are no more instructions to execute. Since each register is an unbounded integer, it can encode whatever the heck you like.*

RAMs

A more sophisticated type of register machine is the **RAM** (Random Access Machine). Instruction sets vary, but usually contain arithmetic, logic, and jumps. Some instruction sets feature three-operands (sometimes called 3AC sets) and might look like this:

Restricted 3AC: Simple Direct Addressing Only

Instruction	Description
LOAD n, r_k	$r_k := n$
COPY r_1, r_j	$r_j := [r_i]$
ADD r_i, r_j, r_k	$r_k := [r_i] + [r_j]$
SUB r_i, r_j, r_k	$r_k := [r_i] - [r_j]$
... and so on for MUL, DIV, POW, AND, OR, LT, LE, EQ, NE, GE, GT, SHL, SHR	
NEG r_i, r_j	$r_j := -[r_i]$
NOT r_i, r_j	$r_j := \neg[r_i]$
Can also include JUMP, JZ, JE, READ, WRITE, HALT from above	

Now we can improve the model a great deal by adding two powerful ideas, (1) we can let our *source* operands be constant values (also known as immediate values), and (2) let our registers be **indirect**. For example:

```
COPY 5, r8      -- with imm operands, no need for LOAD
COPY 13, r5
COPY r8, r2     -- copy contents of r8 into r2
COPY *r8, r2    -- copy contents of r5 into r2
COPY r8, *r2    -- copy contents of r8 into r13
COPY *r8, *r2   -- copy contents of r5 into r13
NEG *r2, r3     -- copy negation of r13 into r3
ADD *r8, 5, r3
SUB r13, *r13, *r2
```

Now sources can be immediates, direct registers, or indirect registers; destinations can be direct or indirect registers (not immediates!). So we can write:

3AC	
Instruction	**Description**
COPY $src, dest$	$\langle dest \rangle := \langle src \rangle$
ADD $src_1, src_2, dest$	$\langle dest \rangle := \langle src_1 \rangle + \langle src_2 \rangle$
SUB $src_1, src_2, dest$	$\langle dest \rangle := \langle src_1 \rangle - \langle src_2 \rangle$
... and so on for MUL, DIV, POW, AND, OR, LT, LE, EQ, NE, GE, GT, SHL, SHR	
NEG $src, dest$	$\langle dest \rangle := -\langle src \rangle$

NOT $src, dest$	$\langle dest \rangle := \neg \langle src \rangle$
JZ src, L	If $\langle src \rangle = 0$ then jump to L
JE src_1, src_2, L	If $\langle src_1 \rangle = \langle src_2 \rangle$ then jump to L
JUMP L	Jump to instruction at address L
READ $dest$	$\langle dest \rangle :=$ next value from input stream
WRITE src	Append $\langle src \rangle$ to output stream
HALT	Stop executing

Here is a GCD program on this machine:

```
// while y != 0:
//   x, y = y, x % y
// print(x)

0 |  READ r0            // x
1 |  READ r1            // y
2 |  JZ r1, 7           // if y is 0, we're done
3 |  COPY r1, r2        // y -> old_y (to "save" it)
4 |  MOD r0, r2, r1     // x % y -> y
5 |  COPY r2, r0        // old_y -> x
6 |  JUMP 2             // back to top of "loop"
7 |  WRITE r0
```

Rather than worrying about the position of instructions, let's use **labels**:

```
// while y != 0:
//   x, y = y, x % y
// print(x)

  READ r0            // x will be in r0
  READ r1            // y will be in r1
start:
  JZ r1, wrapup      // if y is 0, we're done
  COPY r1, r2        // y -> old_y
  MOD r0, r2, r1     // x % old_y -> y
  COPY r2, r0        // old_y -> x
  JUMP start         // back to top of "loop"
wrapup:
  WRITE r0           // print x
```

Another variation requires the target to be one of the sources:

2AC	
Instruction	**Description**
COPY $src, dest$	$\langle dest \rangle := \langle src \rangle$
ADD $src, dest$	$\langle dest \rangle := \langle dest \rangle + \langle src \rangle$
SUB $src, dest$	$\langle dest \rangle := \langle dest \rangle - \langle src \rangle$
... and so on for MUL, DIV, POW, AND, OR, LT, LE, EQ, NE, GE, GT, SHL, SHR	
NEG $dest$	$\langle dest \rangle := -\langle dest \rangle$
NOT $dest$	$\langle dest \rangle := \neg\langle dest \rangle$
JZ src, L	If $\langle src \rangle = 0$ then jump to L
JUMP L	Jump to instruction at address L
READ $dest$	$\langle dest \rangle :=$ next value from input stream
WRITE src	Append $\langle src \rangle$ to output stream
HALT	Stop executing

Code gets a little longer now:

```
// 2AC GCD PROGRAM

// while y != 0:
//   x, y = y, x % y
// print(x)

  READ r0            // x will be in r0
  READ r1            // y will be in r1
start:
  JZ r1, wrapup      // if y is 0, we're done
  COPY r0, r2        // x -> old_x
  COPY r1, r3        // y -> old_y
  MOD r3, r2         // old_x % old_y
  COPY r2, r1        // new y for next iteration
  COPY r3, r0        // new x for next iteration
  JUMP start         // back to top of "loop"
wrapup:
  WRITE r0           // print x
```

We can even make the target register always be implicitly r_0, which, since this is always the destination for all the arithmetic operations, we call the **accumulator**:

1AC (Accumulator Machine)	
Instruction	**Description**
LOAD src	$r_0 := \langle src \rangle$
STORE $dest$	$\langle dest \rangle := [r_0]$
ADD src	$r_0 := [r_0] + \langle src \rangle$
SUB src	$r_0 := [r_0] - \langle src \rangle$
... and so on for MUL, DIV, POW, AND, OR, LT, LE, EQ, NE, GE, GT, SHL, SHR	
NEG	$r_0 := -[r_0]$
NOT	$r_0 := \neg[r_0]$
JZ L	If $[r_0] = 0$ then jump to L
JUMP L	Jump to instruction at address L
READ	$r_0 :=$ next value from input stream
WRITE	Append $[r_0]$ to output stream
HALT	Stop executing

Programs start to get a lot longer with this architecture:

```
// 1AC GCD PROGRAM

// while y != 0:
//   x, y = y, x % y
// print(x)

  READ            // x in r0
  STORE r1        // x in r1
  READ            // y in r0
start:
  JZ wrapup       // if y is 0, we're done
  STORE r2        // old_y saved in r2
  LOAD r1         // x in r0
  STORE r3        // old_x saved in r3
  MOD r2          // x % y in r0
  STORE r1        // new x for next iteration
  LOAD r2         // new y for next iteration
```

```
  JUMP start      // back to top of "loop"
wrapup:
  LOAD r1         // Now x is in r0
  WRITE           // So we can write x
```

We can make all the arguments implicit if we just organize all our registers like a stack:

0AC (Stack Machine)	
Instruction	**Description**
LOAD n	Push n
ADD	Pop into y; pop into x; push $x + y$
SUB	Pop into y; pop into x; push $x - y$
... and so on for MUL, DIV, POW, AND, OR, LT, LE, EQ, NE, GE, GT, SHL, SHR	
NEG	Pop into x; push $-x$
NOT	Pop into x; push $\neg x$
JZ L	Pop into x; If $x = 0$ jump to instruction at address L
JUMP L	Jump to instruction at address L
HALT	Stop executing

Stack machines are cool when executing arithmetic expressions. Work left to right. Example:

```
// Stack machine code for 7 - 3 * 8 ** 1 / 3

LOAD 7    // 7
LOAD 3    // 7 3
LOAD 8    // 7 3 8
LOAD 1    // 7 3 8 1
POW       // 7 3 (8**1)
MUL       // 7 (3*(8**1))
LOAD 3    // 7 (3*(8**1)) 3
DIV       // 7 ((3*(8**1))/3)
SUB       // 7-((3*(8**1))/3)
```

To be able to move things around with a stack architecture, we can add DUP and SWAP instructions, or even have a separate stash of labeled registers for storage.

```
// 0AC GCD PROGRAM

// while y != 0:
//   x, y = y, x % y
// print(x)

// Assuming separate stash of labeled registers for storage,
// BUT all operations can only occur on stack top.
// STORE instruction does a pop.

  READ
  STORE x
  READ
  STORE y
start:
  LOAD y
  JZ wrapup       // if y is 0, we're done
  LOAD x
  LOAD y
  STORE x         // new x is the old y
  LOAD y
  MOD             // x % y
  STORE y         // new y is x % y
  JUMP start      // back to top of "loop"
wrapup:
  LOAD x
  WRITE
```

RASPs

Another kind is called the **RASP** (Random Access Stored Program). Read the Wikipedia article for this one.

> ***Exercise**: What is the historical importance of the RASP model? Why was it created?*

Summary

We've covered:

- ✅ What register machines are and why they are interesting
- ✅ Counter machines
- ✅ The simplest counter machine that is Turing equivalent
- ✅ 3AC RAMs
- ✅ 2AC RAMs
- ✅ 1AC RAMs (accumulator machines)
- ✅ 0AC RAMs (stack machines)
- ✅ RASPs

Automata Theory

Part of understanding computation involves the study of formal models of computation, which have traditionally been called automata.

CONTENTS

Motivation and History • Kinds of Automata • Classic Examples of Automata • Connections To Formal Languages • Connections To Computability Theory • Applications • Where to Lean More • Summary

Motivation and History

The field we now call automata theory arose from many different areas of research, including:

- **Mathematics**, through attempts to formalize the meaning of computation
- **Engineering**, through attempts to characterize (discrete) physical systems
- **Psychology**, through attempts to characterize human thinking and reasoning

These folks all came up with the notion of an **automaton** as a kind of machine, real or abstract, that follows a fixed set of instructions to carry out a step-by-step procedure.

A brief history of the field can be found at Wikipedia.

> ***Exercise**: Skim the Wikipedia article now.*

We want a rigorous theory of automata. Therefore, we will make the following assumptions about the automata that we study:

- A machine must have a **finite description**.
- The input to the machine is represented as **finite string of symbols** that come from **a fixed alphabet**.

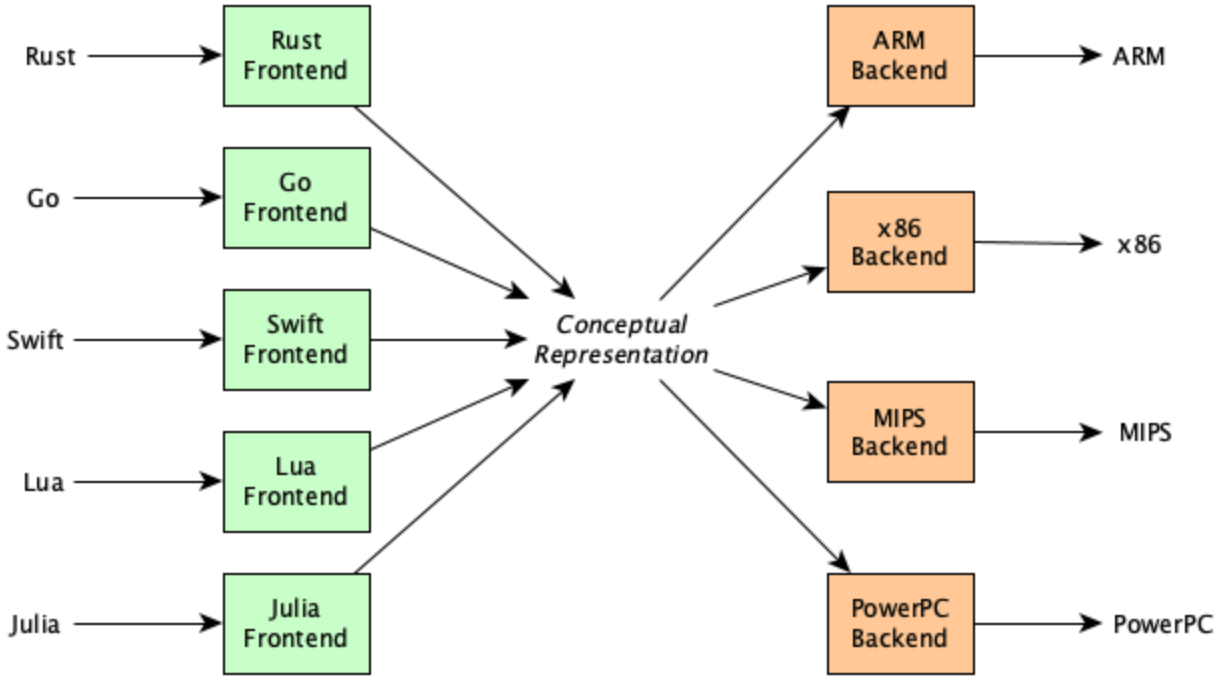
The first assumption is reasonable because a machine that requires an infinite description could not be built, let alone understood. The second is motivated by the fact that it's pretty difficult (impossible?) to conceive of any kind of information that we could not represent with strings of symbols. For example, the following capture quite a variety of information:

- `91772.3e-8`
- `Stay behind the yellow line!`
- `ᐊᐃᖏᓗᐃ ᓄᓇᕐᕕᐊᖅ`
- `/Pecs/Los Angeles/Berlin/Madrid//1 2/3 0/2 2/2 0/3 2///`
- `(* (+ 4 3) (-9 7))`
- `int average(int x, int y) {return (x + y) / 2;}`
- `<dog id="30024"><name>Lisichka</name><age>13</age></dog>`
- `{"type": "dog", "id": "30024", "name": "Lisichka", "age": 13}`
- `∀α β. α⊥β ⇔ (α•β = 0)`
- `1. f3 e5 2. g4 ♕h4++`

> ***Exercise**: Can you think of any information that cannot be represented as a string of symbols? If you can, how would you communicate it to someone else? What would "computing" with such information look like?*

Why create and study a theory of automata? Two reasons:

- It gives us a framework in which we can ask what computation is, what we can compute, what is necessary for computation, and how computations can be carried out.
- It provides insights into how we can design and build both real and abstract machines—and even programming languages! Abstract machines are quite important because they show how to carry out computations independent of any real machine and independent of any real high-level programming language, allowing us build multi-language compiler systems (where the abstract machine serves as a conceptual representation of a program):



Kinds of Automata

An automaton is any device, physical or abstract, that runs by itself (hence the prefix “auto”) to process some information. There should be no limits to your imagination regarding *how* an automaton should or should not look, nor what kinds of internal operations it carries out (as long as it is finitely describable).

People have created many kinds of automata. Some strive to be as minimal as possible (theoretically valuable, but hard to work with in practice); some are much more complex internally (but easier for humans to work with). In terms of technical details, there are at least four dimensions on which we can characterize automata:

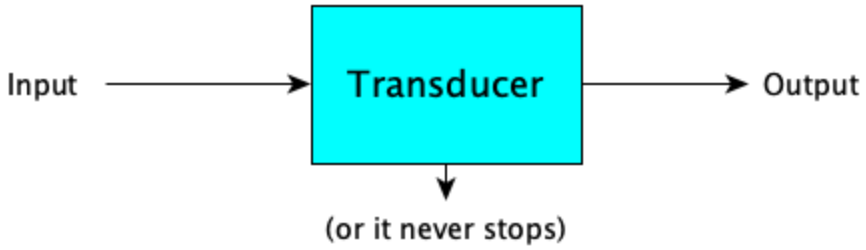
- (Behavior) Is the machine just answering a YES/NO question or computing a result?
- (Operation) How does the machine represent and execute each step of a computation?
- (Memory) Where are the inputs, outputs, and scratch data stored?
- (Architecture) Is the program hard-wired into the machine, or stored in memory with data?

There are more dimensions: synchronous vs. asynchronous, deterministic vs. non-deterministic, definite vs. probabilistic, classical vs. quantum, etc. But as these notes are introductory, we will focus only on the first four dimensions.

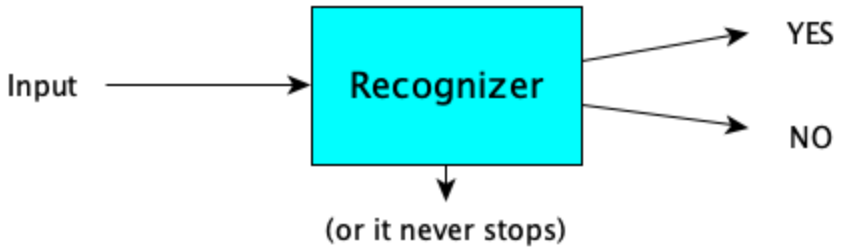
Behavioral Variations

Our first dimension is one of convenience only.

- A **transducer** automaton transforms an input into an output (if it finishes):



- A **recognizer** automaton accepts or rejects its input, by answering a YES-NO question about the input (if it finishes):



  If a recognizer machine M says YES to input w, we say that M **accepts** w.

  A recognizer that always finishes and answers YES or NO (and never loops) is called a **decider**.

Example: A example problem for a transducer might be: Compute the product of this pair of numbers.

Example: A problem for a recognizer might be: Is this number prime?

Operational Variations

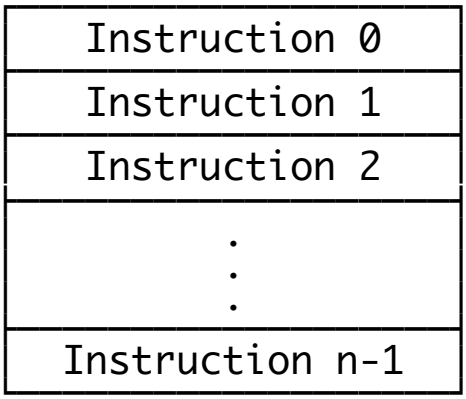
There are variations on how the automata are structured, particularly how their **control unit**, or algorithm, is specified. The main options are with a (1) a state machine, (2) a set of instructions, or (3) a set of rewrite rules.

- A **state machine** automaton performs a computation by moving through **states** that indicate in which phase of the computation one is in. The "program" is basically a **transition table**. The computation ends when you land in a state that doesn't have a transition out of the current state for the current machine configuration. Transition tables come in many forms, depending on the type of state machine, but are generally structured like so:

If you are in this state	and this event occurs	Then perform this action	and move to state

  If there can be more than one action/transition to take from a given state/input configuration, the machine is said to be **non-deterministic**, otherwise it is **deterministic**.

- An **instruction-set machine** has a control made up of a list of instructions. Normally instructions are executed in order, but some machine operands can be references to other instructions (e.g., via labels or by position) to allow conditional and iterative execution. Instructions that directly reference other instructions to explicity change the control flow are called **branches** or **jumps**.



- A **rewrite machine** is an automaton specified with a set of rewrite rules that transform the input to the output. For recognizers, we can have the output be a designated YES or NO symbol. The "program" defined by a rewrite system is something like:

If you see this substring	Rewrite with this substring

> ***Exercise**: Imagine non-deterministic vs. deterministic string rewriting machines. What issues come up in distinguishing these variations?*

Are these really all the same in some sense? Are these distinctions just for human convenience?

***Exercise**: Show how to represent a list-of-instructions machine as a state machine. Show how to represent a string rewriter as a state machine.*

Memory Variations

There's a lot of variety in memory, but two primary kinds are:

- **Tapes**: a tape is sequence of cells, each holding a single symbol. At any point the machine is looking at a **current cell** on the tape. The machine can move left or right only. The tape can have a finite number of cells, or can be infinite in one direction, or infinite in both directions. A machine can have multiple tapes. There are models where tapes are restricted to be read-only, write-only, stack-like (push/pop only), or queue-like (enqueue/dequeue only).



- **Registers**: A machine can have one or more registers. Each register holds some data. Usually in automata theory, a register holds a natural number (nonnegative integer) of unbounded size, but variations allow it to hold only a single symbol, or maybe a floating-point (but not real) number, or even a string. Registers are numbered $r_0, r_1, r_2, \ldots$ and they can be accessed by their number. Some models have a finite number of registers; in others, infinite. There can even be different classes of registers: for example, maybe only certain registers can do arithmetic, maybe only certain registers can be used for indirect addressing, etc.



What else? You can have fun, too, with things like multi-dimensional tapes. (A two-dimensional tape would allow for up, down, left, and right movements.) Use your imagination to build new computing machines.

Architectural Variations

One final variation of interest, applicable to instruction-based automata only, really, is: where are the instructions stored? There are two main architectures:

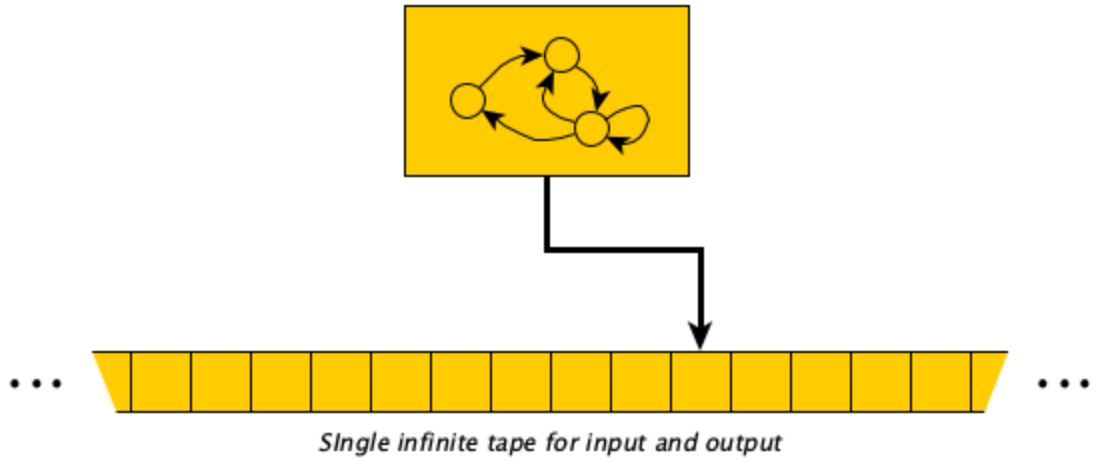
- **Harvard Architecture**: Instructions are stored in a dedicated, read-only instruction unit, separate from data.
- **Von Neumann Architecture**: Instructions are stored together with the data.

Classic Examples of Automata

Turing Machines (TMs)

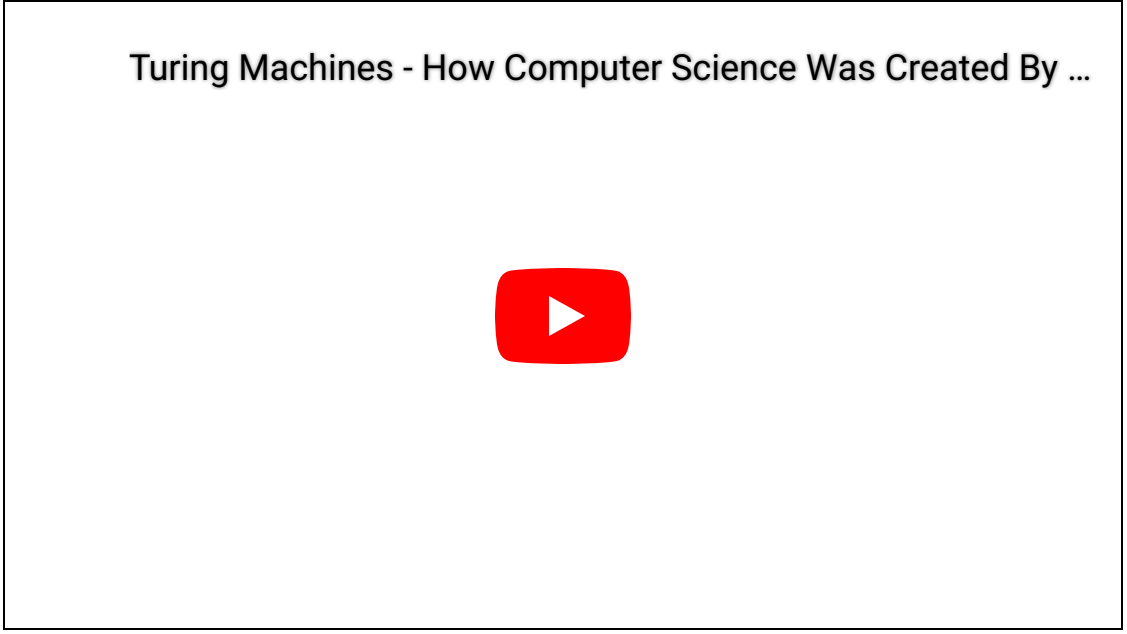
Turing Machines are so historically significant and influential, that I have an entire separate page of notes on them.

But the basic idea is that the input sits on a infinite tape of cells surrounded by blanks. The machine has a head pointing to the first symbol of the input. The machine's control is a finite set of rules that says "If in state q looking at symbol σ, replace the current symbol with σ', move either one cell right or left, and transition to state q'."

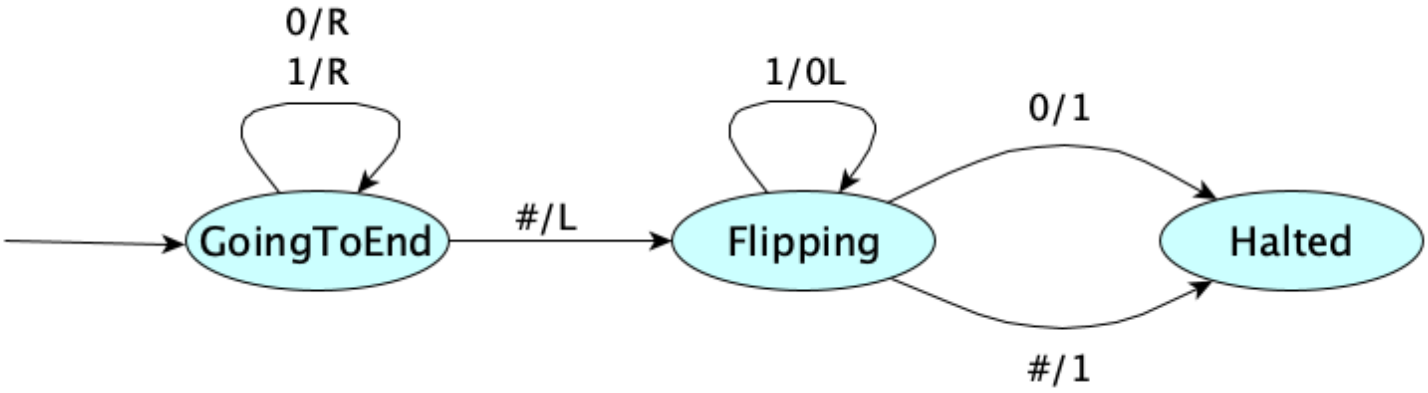


SIngle infinite tape for input and output

Watch Jade's video again:



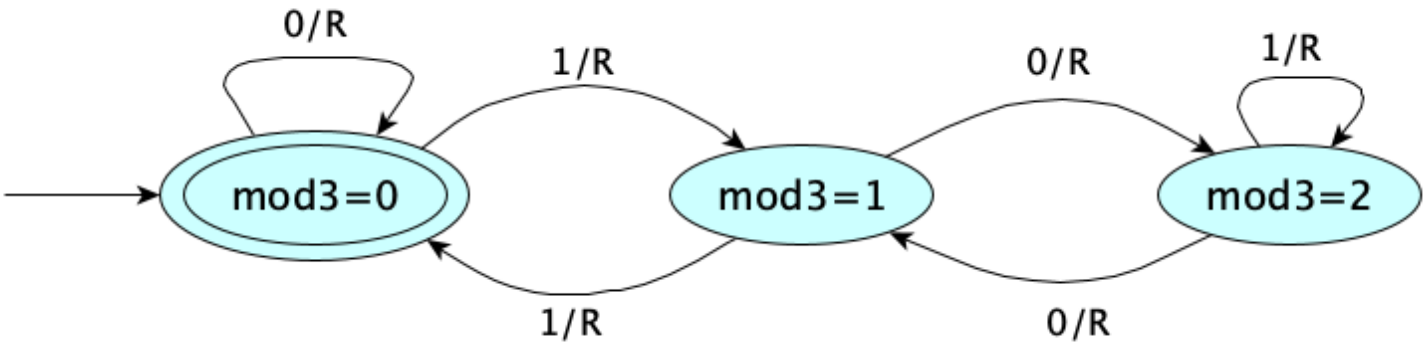
Again, we'll cover Turing Machines in detail later, but for now here's an example Turing Machine (of the transducer variety) to increment a binary number:



CLASSWORK

> Roleplay, with paper and pencil, the human computer of Turing's day sitting at a desk carrying out the task of incrementing the binary number 11010011111. Put yourself in each "mental state" expressed in the above machine.

And here is one (of the recognizer) variety to determine if a given unsigned binary number is divisible by 3:

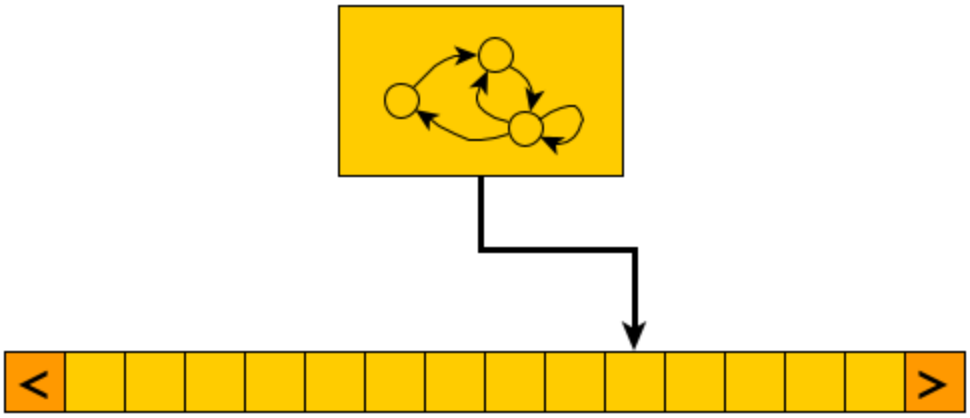


Later, we'll talk about how we came up with these beauties. Not too bad for us, right? But how the heck did Turing just figure all this stuff out to begin with? Pretty amazing, right? Yes, it *is* amazing even though his work was massively influenced by previous work of Gödel and Church and others, which Turing was happy to acknowledge.

> ***Wait, only one tape?***
>
> *Can a Turing Machine have multiple tapes? Sure. Turing was going for simplicity (if not outright minimalism) when we introduced this machine as a model for computation. You can show that any TM with multiple tapes can be simulated with a 1-tape TM, so various other tape-based models of computation can generally be considered kinds of Turing Machines. Some of these we'll see right now.*

Linear Bounded Automata (LBAs)

An LBA is a Turing Machine whose tape is not infinite but rather contains **only** its input with one cell to the left containing an non-replaceable "left-end-marker symbol" and one cell to the right containing a non-replaceable "right-end-marker" symbol.

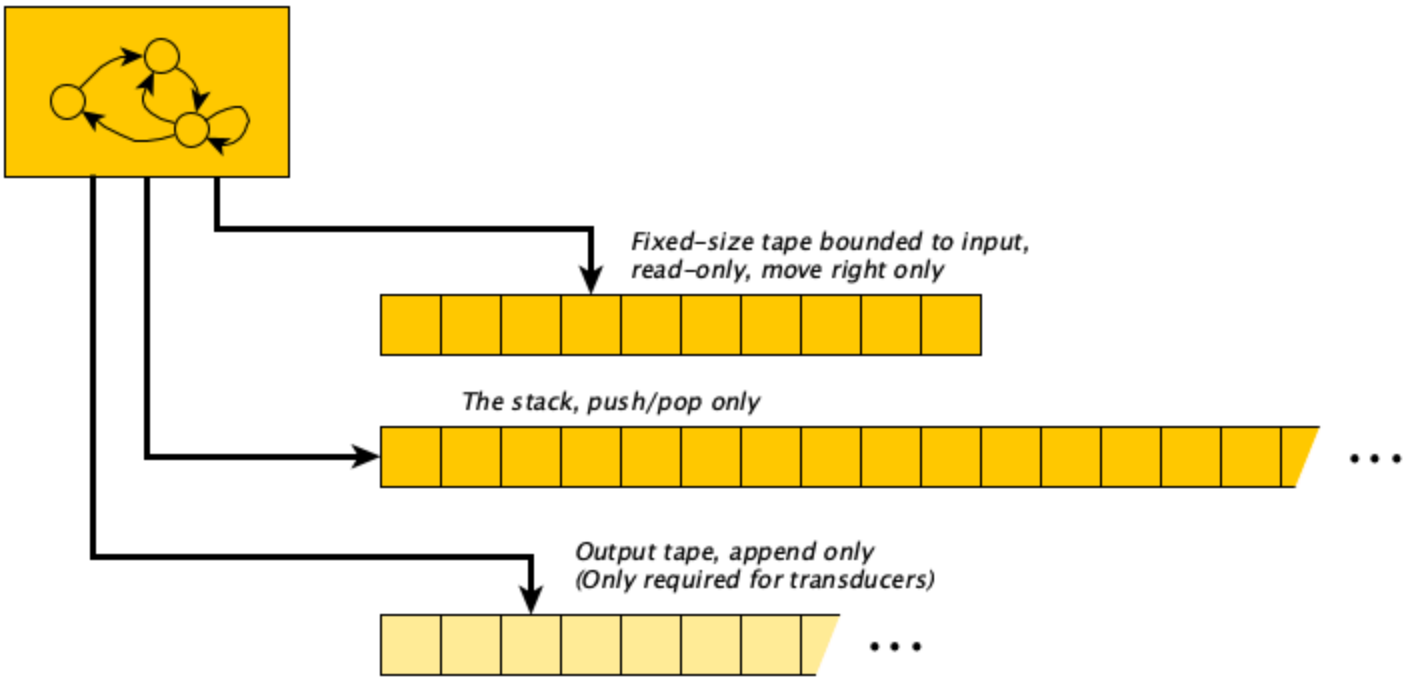


Fixed-size tape bounded to input string plus two cells for endmarkers

It feels a little more realistic, closer to a real computer, because of it having a finite "memory," right?

Pushdown Automata (PDAs)

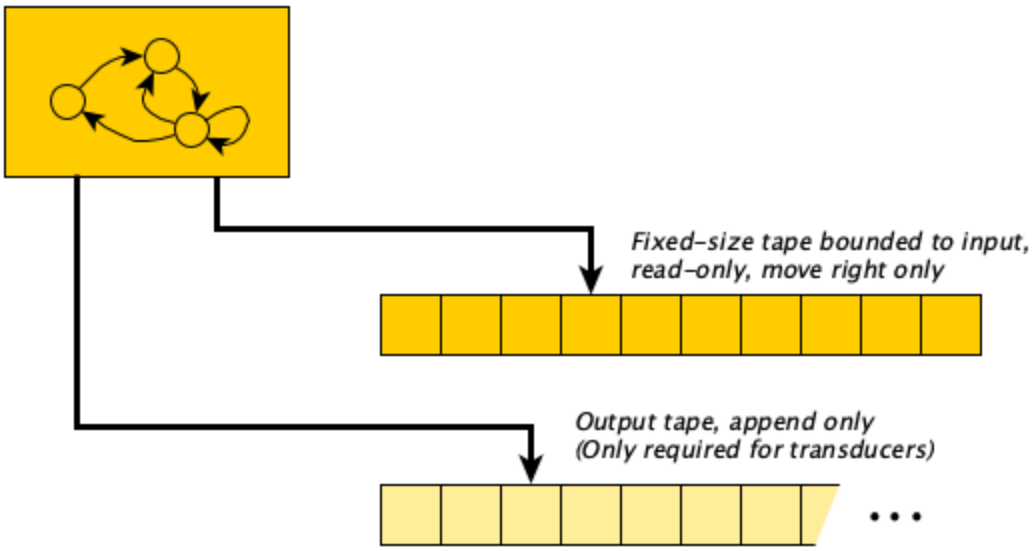
A PDA is an automata with a finite-state control, with a read-only input, and separate stack. At each step you read the next input symbol, and depending on the current top of the stack and the current state, you pop from the stack, push something new, and move to a new state.



***Exercise**: Why would anyone care about a machine like this? (Hint: how efficient are stacks, in terms of the Big-Θ complexity measure you learned about back in the day?)*

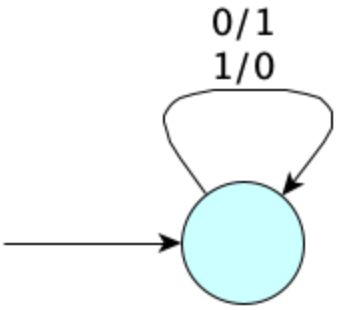
Type-3, or “Finite” Automata (FAs)

A **Type-3 Automaton**, often called just a **Finite Automaton** or **FA**, is a Turing Machine whose input is read-only and that only moves to the right on each step. For transducing FAs, the output is written to a separate output tape that is write-only, initially blank, and only appended to.

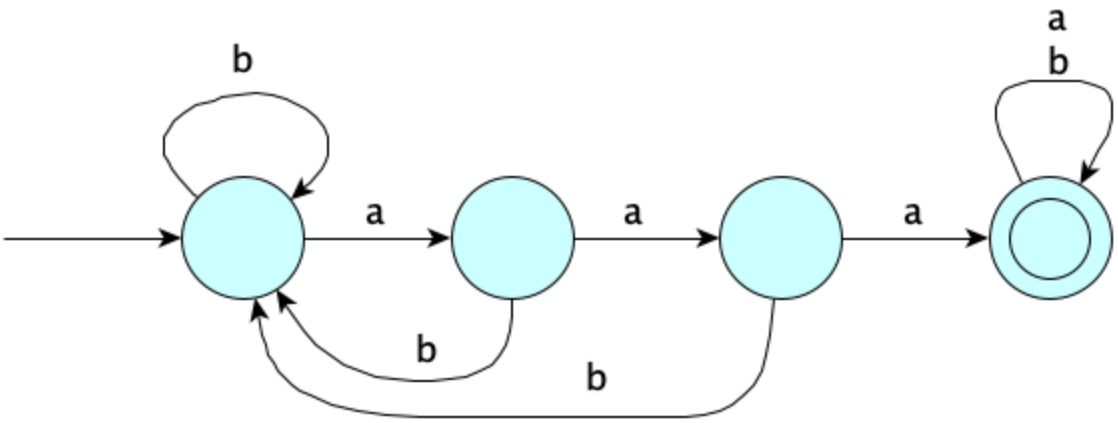


> ***Do not let this terrible name confuse you***
>
> *It is unfortunate that these machines are called finite automata, because Turing Machines have a finite number of states, as do LBAs and PDAs. Maybe it is because of finite memory? No LBAs have finite memory. No one has a good name for this class of automata. Everyone calls them FAs.*
>
> *Perhaps if you never heard of LBAs the name might make sense.*
>
> *By the way, you may sometimes hear them called FSMs for “finite state machines” but that is much, much worse since all reasonable automata have a finite number of states and the acronym is easily confused with the deity of Pastafarianism.*

When describing FAs, we don’t have to worry about blanks, or say whether we are moving left or right! All we need is the input symbol and the output symbol(s). For a first example, here is a transducer FA that computes the one’s complement of its input:



For acceptors, we don't have any output at all, so the machine transitions just have the input symbol. Here's an FA to determine whether its input has three consecutive ***a***'s somewhere:



An FA that writes to its output on a transition is called a **Mealy Machine**. You can also make an FA transducer that outputs a symbol whenever it enters a state; this would be called a **Moore Machine**.

> ***Exercise**: What "kind" of computations do Finite Automata represent?*

Queue Automata

A queue automaton is a TM that, instead of moving left or right at each step, instead always dequeues the symbol at the left and enqueues some other string of symbols on the right.

$\mathcal{P}''$

An interpreter for the $\mathcal{P}''$ language can be considered an automata. This is an exceedingly low-level and minimal notation. Check it out at Wikipedia.

Brainfuck

Like $\mathcal{P}''$, Brainfuck (or Brainf**k as it is commonly known) is a programming language with an explicit machine execution model. It is much more widely known and way easier to understand than $\mathcal{P}''$, as it is less minimal.

The execution model features (1) a fixed program stored on a read-only tape where each cell holds one of eight instructions, (2) a read-only input stream of bytes (you can think of it as a tape), (3) a write-only output stream of bytes (also a tape), and (4) a data tape with a fixed left end and infinite to the right (although really the language specification says at least 30,000 cells). The eight instructions are:

Instruction	Description
+	Increment the byte in the current data cell
−	Decrement the byte in the current data cell
.	Output the byte in the current data cell
,	Read next byte from input and store in current data cell
<	Move left on tape
>	Move right on tape
[	If byte in current data cell is 0, jump to instruction right after the next *matching*]
]	If byte in current data cell is not 0, jump to instruction right after the previous *matching* [

Here's a program in the language. Can you figure out what it does?

```
+++++++[>+++++[>++>+++<<-]<-]>>++.>.
```

▶ Need a hint?

See the Language overview at Esolang, the Wikipedia article, and this collection of example programs.

String Rewriting Systems

We can mechanically compute by applying rules (from a finite set of rules) that allow us to rewrite part of a string into another. We start with the input surrounded by begin and end markers `[` and `]`, and keep rewriting, non-deterministically, there are no more rules to apply.

For example, here's an SRS for incrementing binary numbers:

$0]$	$\to$	$Y1$
$0Y$	$\to$	$Y0$
$1Y$	$\to$	$Y1$
$[Y$	$\to$	ε
$1]$	$\to$	$X0$
$0X$	$\to$	$Y1$
$1X$	$\to$	$X0$
$[X$	$\to$	1

Some example runs:

- $[0] \Rightarrow [Y1 \Rightarrow 1$
- $[1] \Rightarrow [X0 \Rightarrow 10$
- $[101] \Rightarrow [10X0 \Rightarrow [1Y10 \Rightarrow [Y110 \Rightarrow 110$
- $[110010] \Rightarrow [11001Y1 \Rightarrow [1100Y11 \Rightarrow [110Y011 \Rightarrow \cdots \Rightarrow [Y110011 \Rightarrow 110011$
- $[111] \Rightarrow [11X0 \Rightarrow [1X00 \Rightarrow [X000 \Rightarrow 1000$
- $[]$ cannot be reduced, it's an illegal input with an illegal output

How about a formal definition?

> An SRS is a tuple (Σ, R) where:
>
> - Σ is the alphabet of symbols
> - $R \subseteq \Sigma^* \times \Sigma^*$ is the rewriting relation.
>
> The one-step derivation relation $\Rightarrow_R \subseteq \Sigma^* \times \Sigma^*$ is defined as: $xuy \Rightarrow_R xvy$ iff uRv.

SRS Computation

An SRS (Σ, R) *computes output* w' *from input* w *iff*

$$[w] \Longrightarrow_R^* w' \wedge \forall x\, y\, z.\, w' = xyz \supset \nexists u.\, (y, u) \in R$$

We can make a SRS be a recognizer, too. Here is an SRS that determines whether or not its input is in the form $a^n b^n c^n$, for $n \geq 0$:

ab	$\rightarrow$	X
Xb	$\rightarrow$	bX
Xc	$\rightarrow$	ε
$[]$	$\rightarrow$	$\boxed{\text{YES}}$

SRS Acceptance

An SRS (Σ, R) *accepts* w *iff* $[w] \Longrightarrow_R^* \boxed{\text{YES}}$

***Exercise**: Show how* $aaabbbccc$ *is accepted. Show how abbbca is rejected.*

***Exercise**: Give a Turing Machine to decide this language.*

Register Machines

Now let's move from tapes to registers.

A **register machine** has a number of registers $r_0, r_1, r_2, \ldots$, each holding a single (unbounded) natural number. The machine's control is a program, defined as a list of (labeled) instructions. The instructions are normally executed in sequence, but some instructions can cause a "jump" to an arbitrary instruction. The instruction set varies from model to model (of which there exist an infinite number of possibilities). Here's an example of a register machine with a very small instruction set:

Instruction	Description
LOAD n, r_k	$r_k := n$
COPY r_1, r_j	$r_j := [r_i]$
ADD r_i, r_j, r_k	$r_k := [r_i] + [r_j]$
SUB r_i, r_j, r_k	$r_k := [r_i] - [r_j]$
... and so on for MUL, DIV, POW, AND, OR, LT, LE, EQ, NE, GE, GT, SHL, SHR	
NEG r_i, r_j	$r_j := -[r_i]$
NOT r_i, r_j	$r_j := \neg[r_i]$

Can also include JUMP, JZ, JE, READ, WRITE, HALT from above

Register machines are rather important, as they are pretty much the basis for most "real" machines today. We'll discuss them in more detail later.

Lambda Calculus Interpreters

Any lambda calculus function is essentially a program. We tend to think of the λ-calculus as a formalism for modeling computation (and computable functions) in general, but one could, in theory, build hardware to carry out the operations of the calculus, so it's fun to consider it along with more classical automata.

An early one was the SECD Machine.

> ***Exercise**: Study this machine.*

μ-Recursive Functions

The μ-Recursive Functions, also known as the **General Recursive Functions**, or the **Partial Recursive Functions**, are those functions defined by a small number of rules. They allow computations in "bigger steps" than the λ-Calculus.

And More!

Yes, people have created probabilistic automata and quantum automata, too.

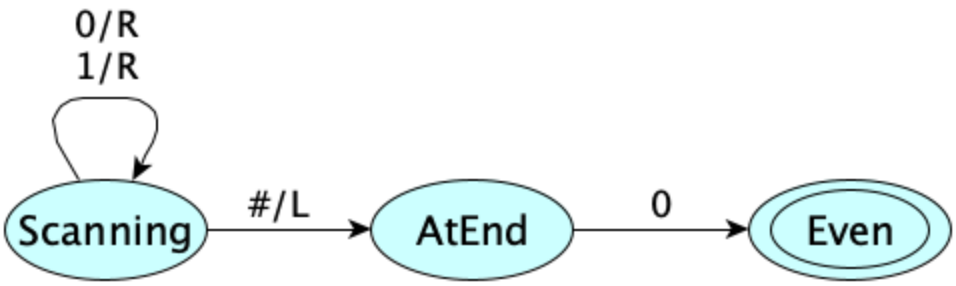
Connections To Formal Languages

A recognizer automata actually defines a formal language! That is, the language of machine M is:

$$L(M) = \{w \mid M \text{ accepts } w\}$$

Nice! This means a formal language can be defined by a grammar or by a machine. This is philosophically interesting, if not profound. It feels like there is a kind of symmetry going here.

How about an example, to understand the important concepts here, and to get the vocabulary right? Let M be the Turing Machine given by the following state diagram:



When run on any non-blank input of binary digits, the machine will end in the accept state iff the input ends with a 0. Otherwise it will get stuck. Therefore we say:

- M **accepts the string** 110100100
- M **recognizes the language** $\{w \in \{0,1\}^* \mid w \text{ is an even binary integer}\}$
- M **accepts the language** $\{w \in \{0,1\}^* \mid w \text{ is an even binary integer}\}$
- M **decides the language** $\{w \in \{0,1\}^* \mid w \text{ is an even binary integer}\}$
- $L(M) = \{w \in \{0,1\}^* \mid w \text{ is an even binary integer}\}$

> ***Accepting vs. Deciding***
>
> *In tne example above, the machine M did indeed both accept and decide the language of even binary integers. However, there do exist languages that are recognizable by a TM but not decidable by any TM. The difference between accepting and deciding is part of computation theory, and we'll study this later.*

Connections To Computability Theory

We can characterize different automata by the classes of functions they can compute, or equivalently, the languages they can recognize. For example Turing Machines can recognize a much larger class of languages than can Type-3 Machines.

So we can rank automata in increasing "power." In the following table, the class of languages/problems in each row is a proper subset of the following row:

Automata	Problems that can be solved / Languages that can recognized	Example
Loop-free FA	Finite Languages. Languages with a finite number of strings, you can basically just list them all out. Here the FA is simply a directed-acyclic graph.	$\{a, b\}$
FA (Type-3 Machine)	Regular Languages. You can recognize these without any external memory whatsoever. And you can process them in linear time.	a^*
Deterministic PDA (DPDA)	Deterministic Context-Free Languages. An interesting subset of the context-free languages.	$a^n b^n$
Nondeterministic PDA (NPDA)	Context-Free Languages. It's quite interesting, and not really too hard to prove, that the NPDAs recognize exactly the class of CFLs, which are the languages generated by a Context-Free (Type-2) Chomsky Grammar. This is a fairly natural class of languages.	ww^R
LBA	Type-1 Languages. Bounding the tape of a TM reduces the computing power significantly!	$a^n b^n c^n$
TM that always halts	Decidable Languages. Also known as the Recursive Languages, **R**.	LBA_{diag}
TM	Type-0 Languages. Recursively Enumerable Languages, **RE**. For languages that are in RE but not in R, a TM can be found that will accept (say YES) every string in the language, but will not be able to halt and say NO for all strings not in the language.	H
—	Finitely Describable Languages. Any language that can be finitely described, whether it can be computationally recognized or not.	$\overline{H}$

Can any automata compute a larger class of functions than a Turing Machine? We don't think so. In fact, the following are all equal in computing power:

- Turing Machines
- Deterministic TMs

- One-way infinite tape TMs
- Multi-tape TMs
- Multi-dimensional TMs (e.g., can move N, E, S, W)
- Queue Automata
- NPDAs with two stacks
- Register Machines
- RAMs
- RASPs
- The μ-recursive functions
- Post Systems
- Phrase-Structure (Type-0) Grammars
- The Lambda Calculus

The fact that these all are essentially the same thing, and all attempts at making them more powerful don't actually increase their power, led to the **Church-Turing Thesis**, namely, "The Turing Machine captures exactly we intuitively think of as computable."

> ***Exercise**: Read about the Chruch-Turing Thesis at the Stanford Encyclopedia of Philosophy and at Wikipedia.*

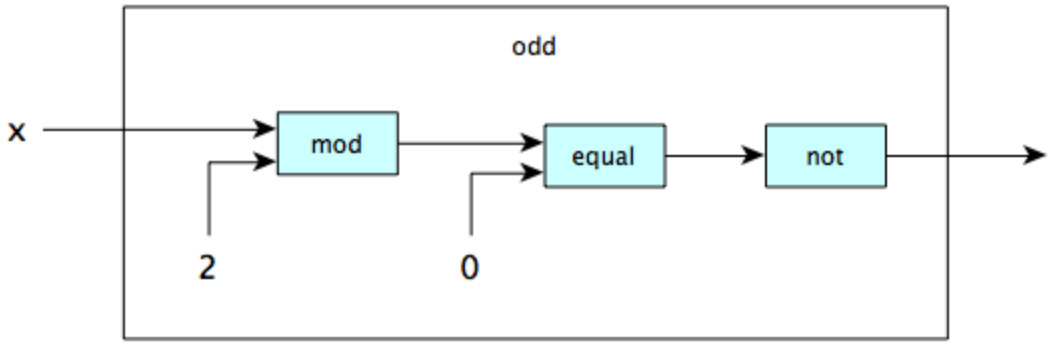
Applications

One of the reasons we have theories is to apply them. (Remember, a theory is an organized body of knowledge with explanatory and predictive powers.) What are some applications of automata?

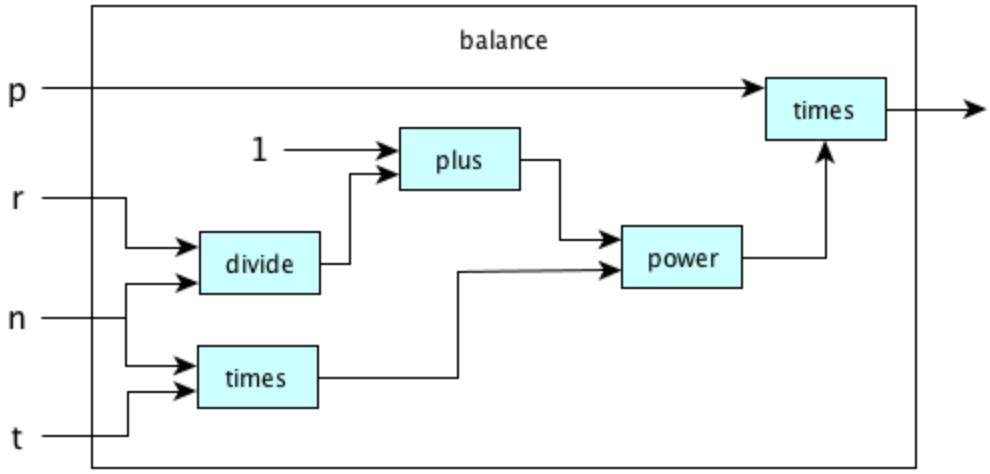
Type-3 machines are used in Lexical Analysis, (extracting "words" from a string of characters), an important operation in text processing, and especially in compiler writing.

Type-2 machines are used in Parsing, uncovering the structure of a string of symbols, another hugely important and common operation.

We can see the relationship between automata and functions, through pictures like:



ant this one:



These pictures are good for humans to build a conceptual understanding of programming by combining functions.

Where to Lean More

Check out the following sources for more information:

- Wikipedia article on Automata Theory
- A Short Introduction by some folks at Stanford in 2004
- The Sipser Theory of Computation Book

Summary

We've covered:

- ✅ What automata theory is
- ✅ Some history of automata theory
- ✅ Various types of automata
- ✅ Relationship of automata theory with formal language theory
- ✅ Relationship of automata theory to computability theory